AutoJudge : Predict Programming Problem Difficulty

The aim is to develop AutoJudge, which is a system predicting both Difficulty Level (Easy, Medium, Hard) and Difficulty Scores of programming problems based only on textual input fields (Problem Description, Input Description, Output Description). The system uses both TF-IDF Features and 14 manually designed auxiliary features to develop models corresponding to both Classification and Regression problems. A **Streamlit** Web Interface is used to show real-time performance.

1. **Introduction**

Coding sites allow coding tasks to be labeled according to their difficulty levels. Manually labeling tasks for coding is a subjective process. AutoJudge uses Natural Language Processing (NLP) techniques as well as Classical Machine Learning algorithms. The final output will consist of:

(a)Trained models (best_clf_model.pkl & best_reg_model.pkl),

(b) Streamlit application (app.py) for prediction tasks,

2. **Files Used**

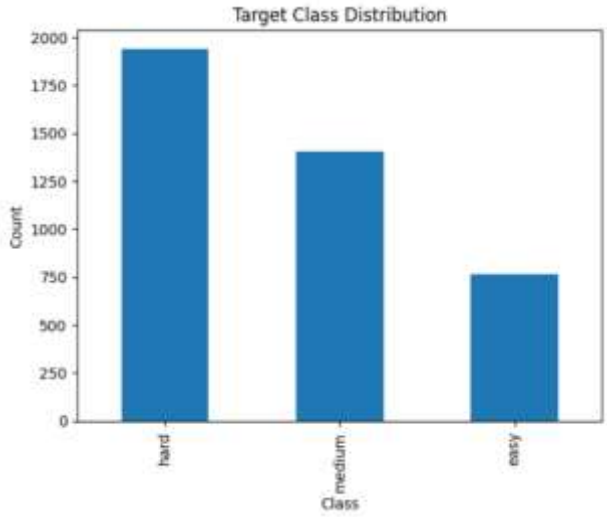
app.py — Streamlit web interface for demonstrating the feature extraction process and the model loading process.

Model.ipynb – EDA on dataset, feature engineering, feature extraction, model training, evaluation on testing dataset.

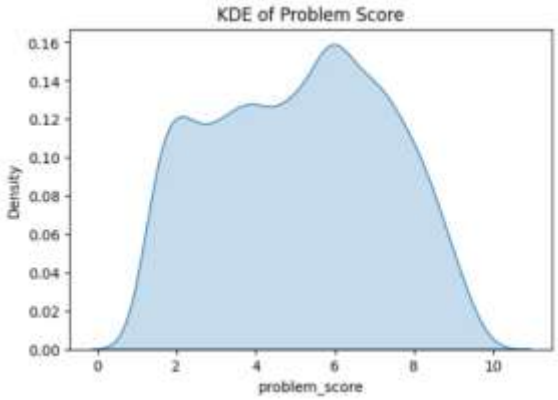
3. **Dataset**

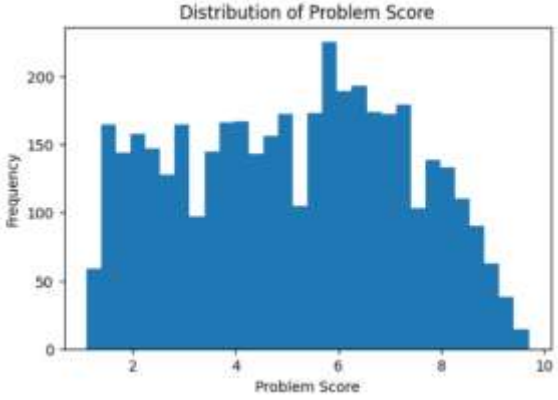
Each data set has: title, description, input_description, output_description, problem_class (Easy/Medium/Hard), and problem_score (numeric). The data set given in this project is considered the sole source for these data.

4.**EDA**



Class imbalance, with the Hard class dominating the dataset and Easy being the least represented.





The uneven spread across score ranges confirms dataset imbalance in difficulty levels

5. **Data preprocessing**

All these steps are implemented in the notebook and correspond to the code used in training and in the inference process in `app.py`.

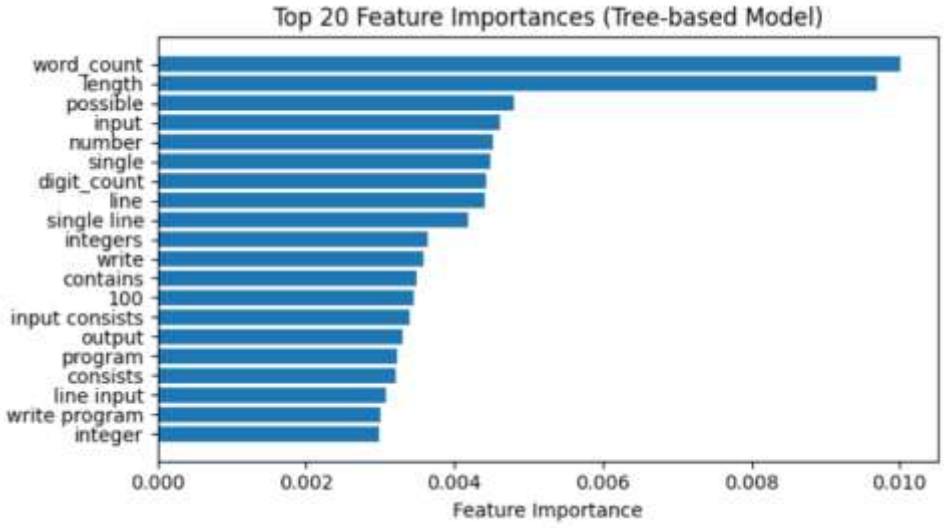
- Connecting text fields: text = description + ' ' + input_description + ' ' + output_description.
- Lowercase letters and basic whitespace fix.
- Missing value handling: use an empty string to replace any missing text in concatenation.
- TF-IDF Vectorizer: fit TfidfVectorizer(max_features=20000, ngram_range=(1,2),

- Helper feature extraction: refer to the build_aux_features function in app.py. (14 features: size, word count, digital count, symbol count, 10 keyword counts). Scaled using StandardScaler. Pickled as aux_scaler.pkl.
- Train/test split: use train_test_split
- To train models, perform classification and regression on TF-IDF features and scaled aux features concatenated together (TF-IDF + scaled aux features); save best models as best_clf_model.pkl and best_reg_model.pkl.

6. **Feature engineering**

TF-IDF features:
- Captures importance of words.
- Use unigrams + bigrams.
- Maintain a limit on vocabulary size (20k) for tradeoff between performance and inference

Auxiliary features correspond to the supporting tasks. They add to the primary task but should not overlap



7. **Models and Training**

Handled Class imbalance and Multi-class classification:
RandomForestClassifier: chose the one that performs better on validation set data. Also, serialize label encoder and dump it to label_encoder.pkl.

Regression: use RandomForestRegressor/ GradientBoostingRegressor when there is a nonlinear relationship, while tuning the hyperparamters with GridSearchCV.

Saved artifacts (used in app.py): best_clf_model.pkl, best_reg_model.pkl, tf

8. **Evaluation metrics**

Train: 80% split, test: 20% split

Classification Metrics: accuracy, precision, recall, F1, confusion matrix.

	Model	Train Accuracy	Train F1	Train Precision	Train Recall	Test Accuracy	Test F1	Test Precision	Test Recall
0	Logistic	0.812709	0.808658	0.823801	0.812709	0.530984	0.514347	0.518110	0.530984
1	DecisionTree	0.698997	0.678462	0.810871	0.698997	0.479951	0.418478	0.412227	0.479951
2	RandomForest	0.979933	0.979679	0.980671	0.979933	0.545565	0.461369	0.508203	0.545565
3	LinearSVC	0.982670	0.982649	0.982672	0.982670	0.492102	0.496677	0.503400	0.492102

Regression: MAE, RMSE, R2 score, MSE

	Model	Train MAE	Train MSE	Train RMSE	Train R2	Test MAE	Test MSE	Test RMSE	Test R2
0	Linear	0.001838	0.001465	0.038274	0.999689	3.097258	14.933293	3.864362	-2.110944
1	DecisionTree	1.386886	3.029748	1.740617	0.357185	1.924408	5.465880	2.337922	-0.138667
2	RandomForest	0.704661	0.716441	0.846428	0.847994	1.687198	4.157980	2.039112	0.133798
3	GradientBoosting	1.100869	1.693775	1.301451	0.640636	1.683727	4.108628	2.026975	0.144079
4	KNN	1.500772	3.293692	1.814853	0.301185	1.829429	4.882176	2.209565	-0.017068
5	SVR	1.762243	4.305473	2.074963	0.086517	1.817457	4.662496	2.159281	0.028696

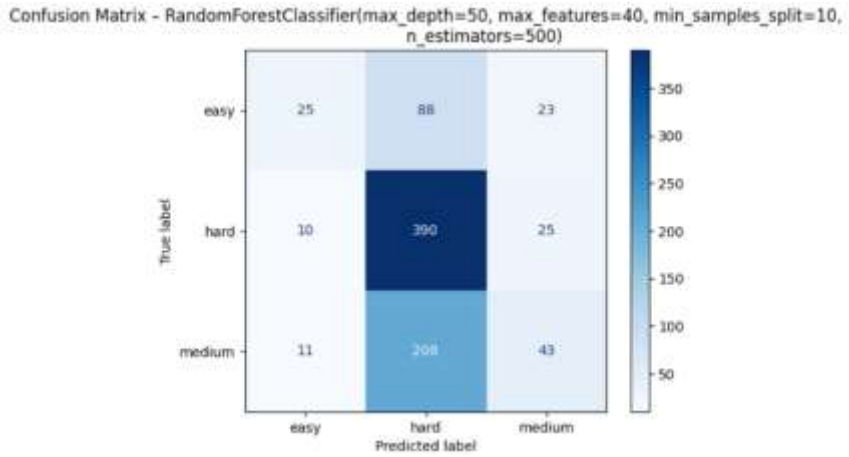
Cross-validation: 5-fold CV for hyperparameter search.

Reproducibility: random_state = 42 for all

9. **Results** →

Classification metrics

Test accuracy: 0.545
Test F-1 score: 0.461
Confusion matrix:



The model correctly identifies **Hard** problems with high recall, while **Easy and Medium classes are frequently misclassified as Hard**, indicating a bias toward predicting higher difficulty levels.

Per-class F1 scores: table (Easy / Medium / Hard).

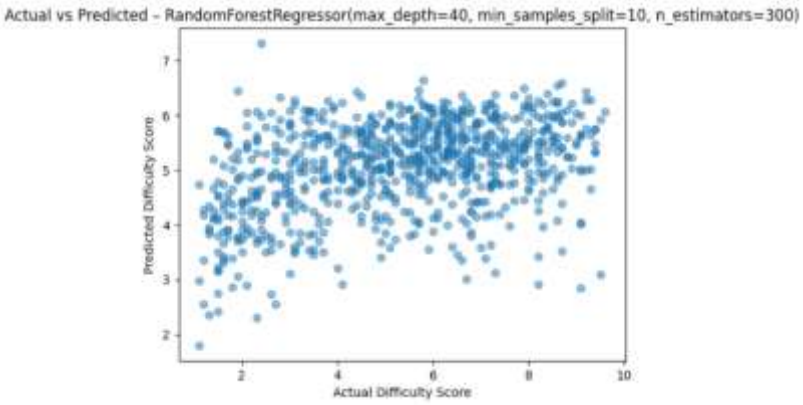
Per-class Precision, Recall, and F1-score

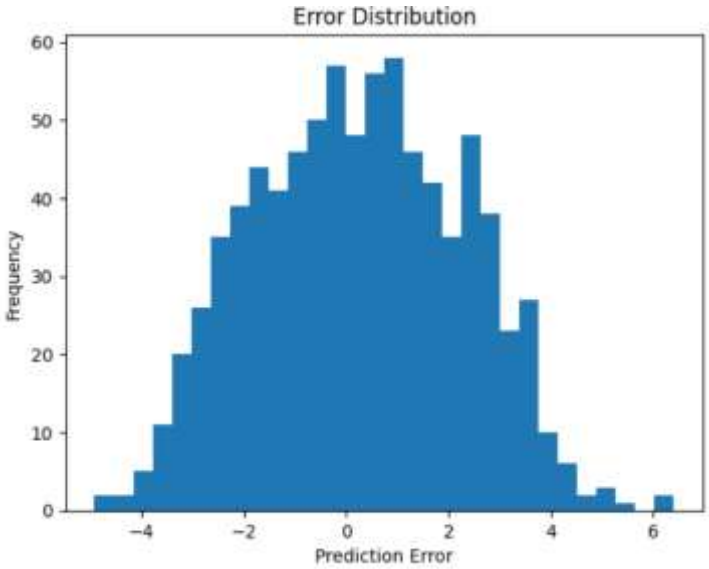
	precision	recall	f1-score
easy	0.543	0.184	0.275
hard	0.569	0.918	0.702
medium	0.473	0.164	0.244

Regression metrics

Test MAE: 1.687
Test MSE: 4.157
Test RMSE: 2.039

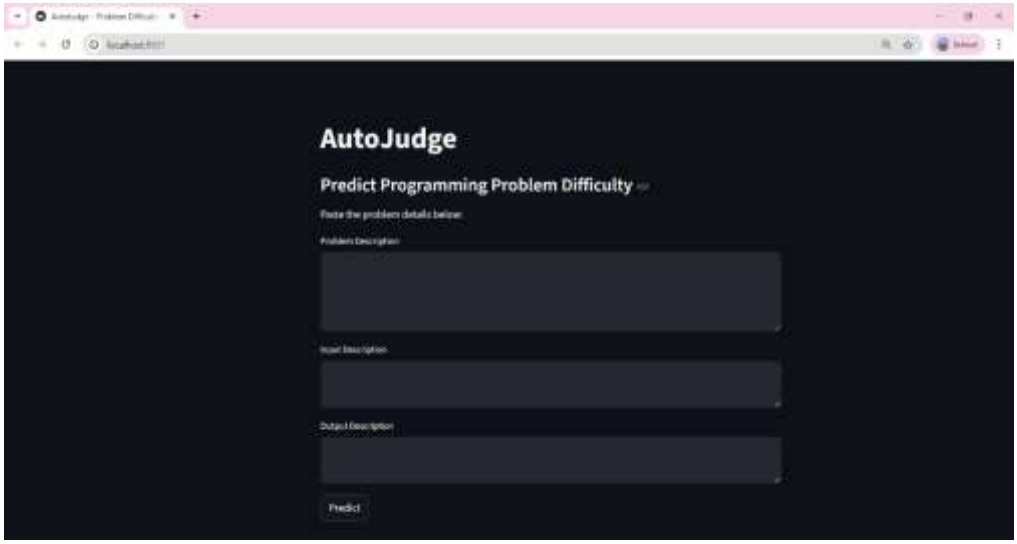
Scatter plot: Predicted vs Actual difficulty scores



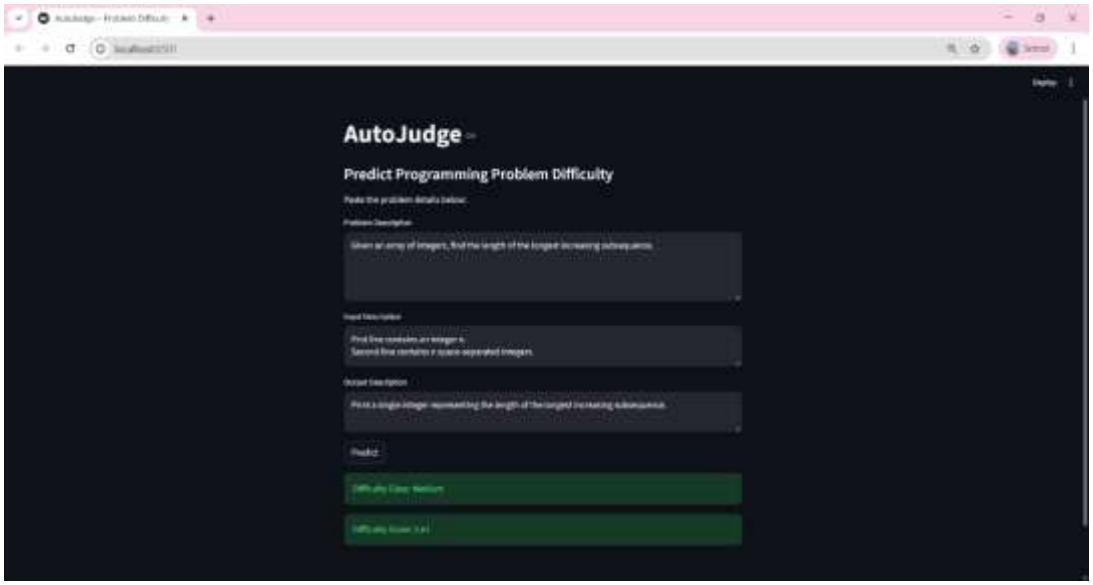


Errors are centered near zero, indicating low systematic bias, while wider tails reflect larger deviations for complex or ambiguous problem description

10. **Streamlit Web Interface**



Streamlit interface for interactive difficulty class and score prediction using trained AutoJudge models.



11. **Limitations**

- **High-Dimensional Sparse Features (TF-IDF)**
  Using ~5,000+ TF-IDF features creates a sparse, high-dimensional input space, increasing memory usage and slowing down model training and inference.
- **Computational Cost of Ensemble Models**
  Tree-based models such as **Random Forest** and **Gradient Boosting** require training hundreds of trees, making them computationally expensive and time-consuming.
- **Slow Hyperparameter Tuning**
  Exhaustive tuning using **GridSearchCV** significantly increases training time, especially for ensemble models with multiple hyperparameters.
- **Limited Real-Time Scalability**
  The combined TF-IDF and auxiliary feature pipeline introduces noticeable latency in the Streamlit application when processing longer problem descriptions.
- **Model Expressiveness Limitation**
  TF-IDF-based models lack deep semantic understanding of text, limiting performance on problems where difficulty depends on implicit reasoning rather than explicit keywords.

12.**Conclusion**

This project has accomplished the development of the AutoJudge, a predicting engine that predicts the difficulty level (Easy, Medium, Hard) and a numeric difficulty value based on the textual description alone. The engine combines the TF-IDF transformation of the input texts with 14 hand-crafted features. The engine is a very good example of feature engineering.

The employment of ensemble learning using machine learning models allowed for effective completion of both classification and regression tasks. In addition to these, evaluation metrics as well as the use of the confusion matrix helped to interpret the performance of these tasks when dealing with class imbalance. The Streamlit-powered website helps to showcase the effectiveness of this system as it allows for real-time prediction.

Despite being faced with high-dimensional sparse features and computationally complex models, the project provides a good baseline for automated difficulty evaluation using traditional NLP methods. In general, the significance of AutoJudge is that it has demonstrated the possibility of replacing manual labeling of difficulties with a data-driven approach.